

NYCU-EE IC LAB – FALL 2026

Lab04 Exercise

Design: Simplified FlashAttention Forward Engine

Data Preparation

1. Extract files from TA's directory:

```
% tar xvf ~iclabTA01/Lab04_2026F.tar
```

2. The extracted LAB directory contains:

- a. **00_TESTBED**
- b. **01_RTL**
- c. **02_SYN**
- d. **03_GATE**
- e. **09_SUBMIT**

Design Description

FlashAttention is an efficient implementation of the self-attention mechanism designed to reduce memory usage and data movement.

Traditional attention computes and stores the entire attention score matrix before performing Softmax and multiplying it with the Value (V) matrix. As the number of tokens increases, the size of the attention score matrix grows rapidly.

FlashAttention solves this problem by dividing the Query (Q), Key (K), and Value (V) matrices into smaller blocks. Each block is processed sequentially using online softmax, so the complete score and softmax matrices do not need to be stored.

The Picture is from paper: Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. For this design, we're going to design a simplified two-head FlashAttention module with 16 tokens, four dimensions of each word embedding, and a block size of two in Verilog code.

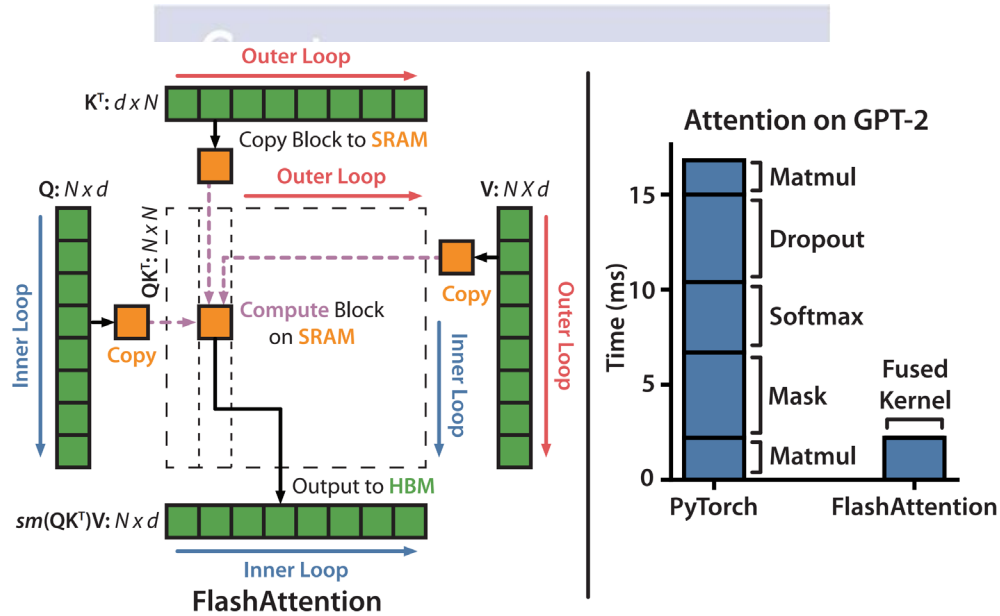
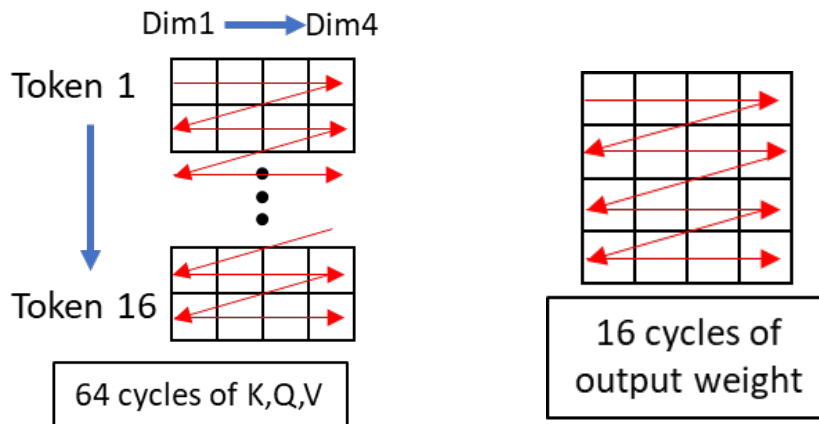


Fig 1. **Left:** FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix
Right: Speedup over the PyTorch implementation of attention on GPT-2.

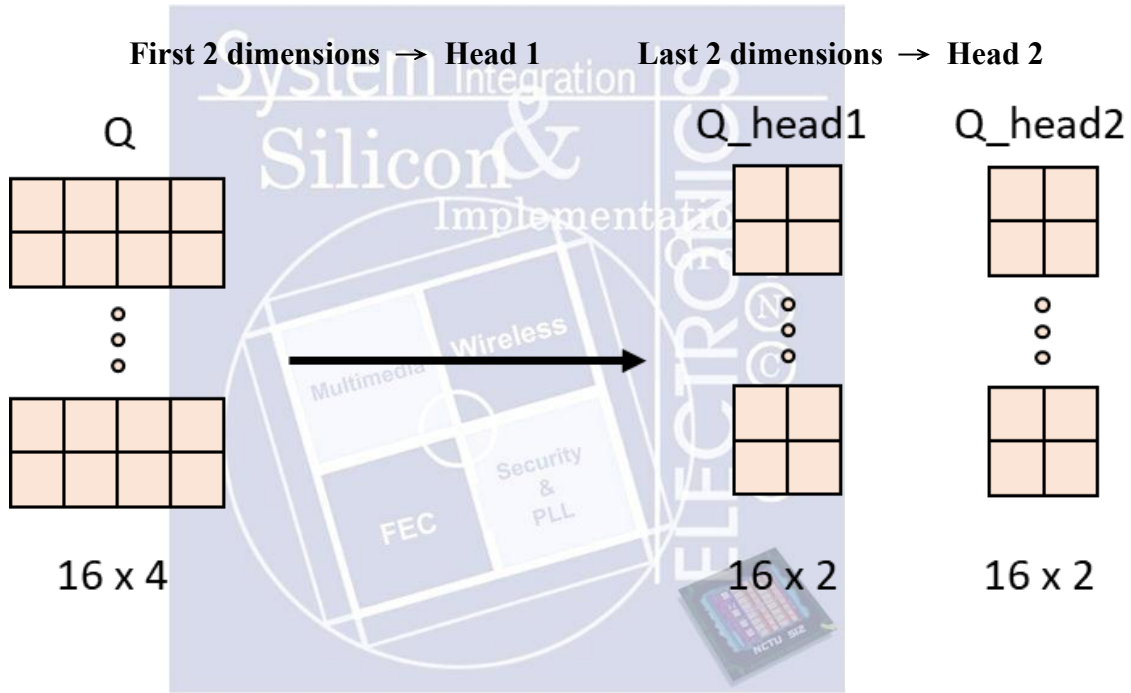
● Description of input signals

The in_valid is pulled high for 64 cycles, and you will get 64 cycles of k, q, v, and the out_weight for output linear layer (4 x 4 cycles)

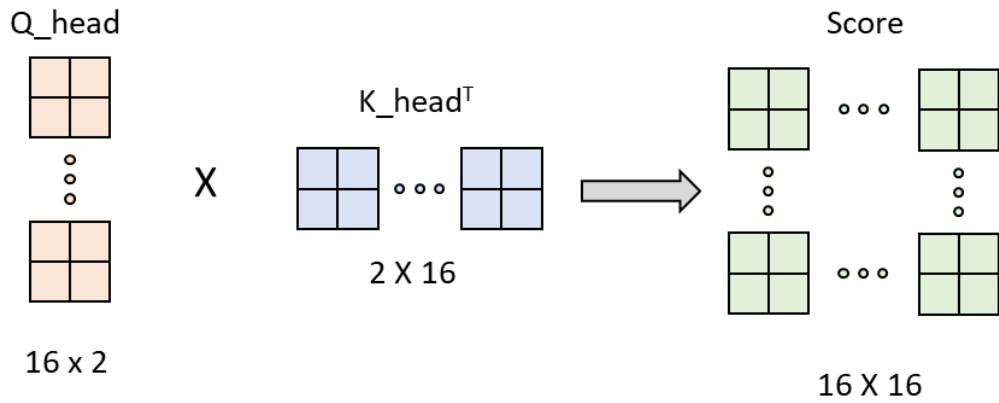


In FlashAttention, each token is associated with three weight matrices: Key (K), Query (Q), and Value (V). The computation follows these steps:

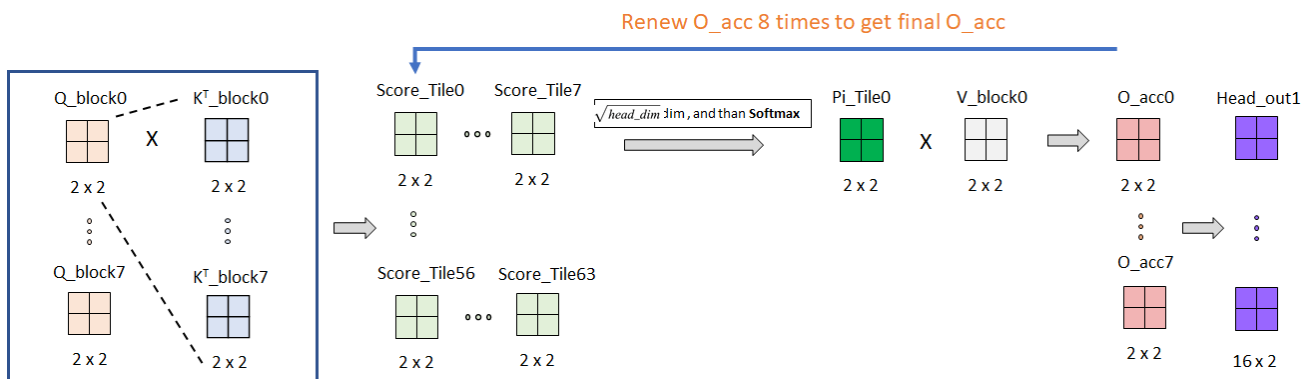
1. Each token has four dimensions. Since two heads are used, the dimensions are split as follows:



2. Score: compute the attention scores by $Q \times K^T$



3. However, To reduce memory usage and data movement caused by storing the complete attention score matrix, we use FlashAttention to divide the attention computation into smaller blocks and process them sequentially

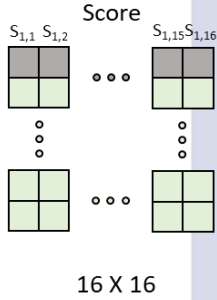


Head Scaling: to make sure the softmax_res in a reasonable range, each number will be divided by $\sqrt{\text{head_dim}}$, which is $\sqrt{2}$

```
parameter square_root_2 = 32'b0011111101101010000010011110011;
```

Softmax:

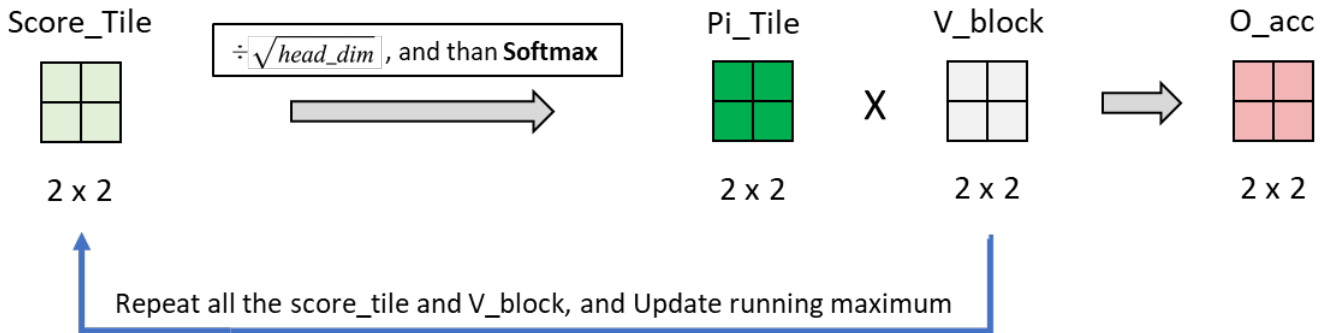
For example:



$$P_{ij} = \frac{e^{S_{ij}-m_i}}{\sum_{j=1}^N e^{S_{ij}-m_i}}$$

$$m_i = \max S_{ij}$$

Because it involves max(score), whenever a new score_tile appears, if max(score) changes, the old score_tile in the same row must be updated.



Online softmax state:

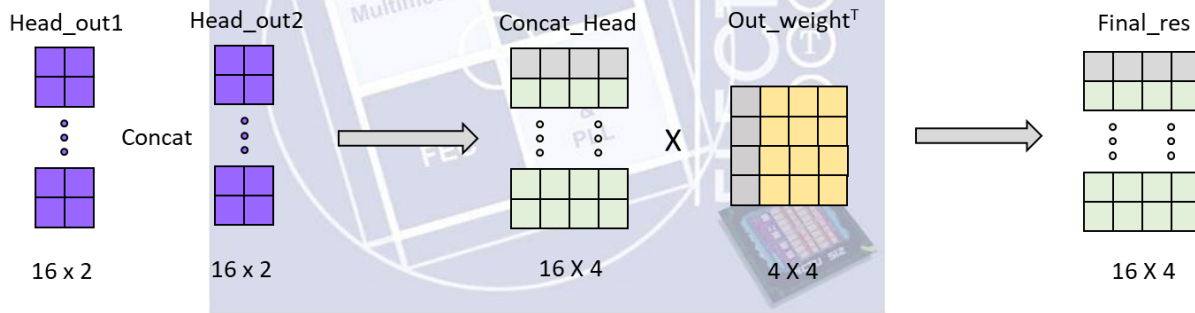
$$m_{\text{new}} = \max(m_{\text{old}}, \text{rowmax}(S_{\text{tile}}))$$

$$l_{\text{new}} = e^{m_{\text{old}}-m_{\text{new}}} l_{\text{old}} + \text{rowsum}(e^{S_{\text{tile}}-m_{\text{new}}})$$

$$O_{\text{acc,new}} = e^{m_{\text{old}}-m_{\text{new}}} O_{\text{acc,old}} + e^{S_{\text{tile}}-m_{\text{new}}} V_{\text{tile}}$$

$$\text{Head_Out} = O_{\text{acc}}/l$$

4. Concatenation: concat the head1 to the first two dim, and head2 for the last 2 dim
5. Linear Projection: Pass through a **linear layer** to project back to the original embedding space.
6. Output your Linear Projection results in raster scan order, so you have to output for 64 cycles.



Inputs and Outputs

The following are the definitions of input signals

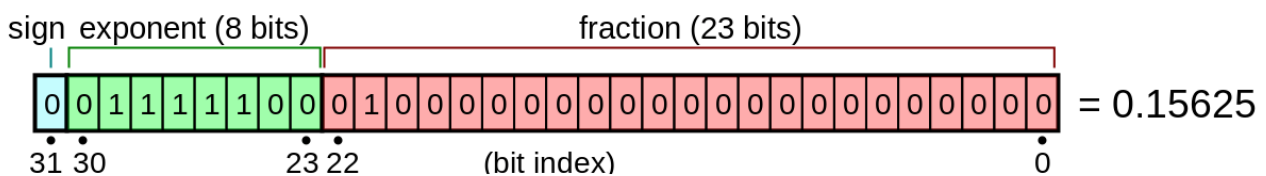
Input Signals	Bit Width	Definition
clk	1	Clock.
rst_n	1	Asynchronous active-low reset.
in_valid	1	High when all input is valid.
K	32	The array signals which sent in raster ordering. The arithmetic representation follows the IEEE-754 floating number format. (Range: $\mp 0.5 \sim 255.0$)
Q	32	The array signals which sent in raster ordering. The arithmetic representation follows the IEEE-754 floating number format. (Range: $\mp 0 \sim 0.5$)
V	32	The array signals which sent in raster ordering. The arithmetic representation follows the IEEE-754 floating number format. (Range: $\mp 0 \sim 0.5$)

Out_weight	32	The Out_weight signals which sent in raster ordering. The arithmetic representation follows the IEEE-754 floating number format. (Range: $\mp 0 \sim 0.5$)
------------	----	---

The following are the definitions of output signals

Output Signals	Bit Width	Definition
out_valid	1	High when out is valid.
out	32	The output signals which will be sent in raster scan order for the final output. The arithmetic representation follows the IEEE-754 floating number format.

Each time you output the result, our pattern will check the correctness of it. Basically, if you follow the formulas and use IEEE floating point number IP, you should get same result as our answer. **However, we release the constraint; you may have an error under $10e-7$ for the result after converting to float number. This means that we will convert your output from binary format into real float number, and compare with our answer. Error will be calculated by 'golden-answer' and get its absolute value. If the error is higher than the value, you will fail this lab.**



$$\text{sign} = +1$$

$$\text{exponent} = (-127) + 124 = -3$$

$$\text{fraction} = 1 + 2^{-2} = 1.25$$

$$\text{value} = (+1) \times 1.25 \times 2^{-3} = +0.15625$$

Binary form: 00111101101100101010000001000101

IEEE floating number: 0.08721975

nWave:

8.72198e-02

nWave will round the number for display, but the computation will not be affected. The following numbers are all from nWave, thus the computations are performed by IPs in Verilog.

※ $e-02 = 10^{-2}$

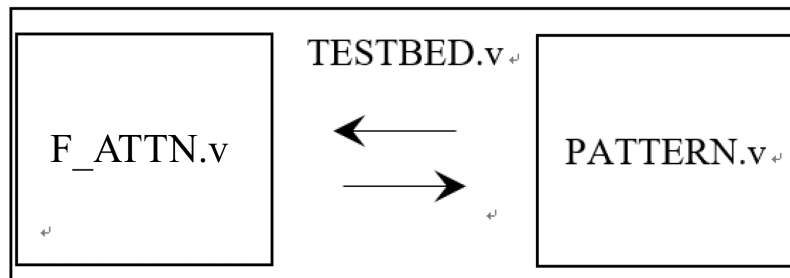
1. The input signal **Out_weight** is delivered in raster scan order for **16 cycles** continuously. When **in_valid** is low, input is tied to unknown state.
2. The input signal **K** is delivered in raster scan order for **64 cycles** continuously.
3. The input signal **Q** is delivered in raster scan order for **64 cycles** continuously.
4. The input signal **V** is delivered in raster scan order for **64 cycles** continuously.
5. All input signals are synchronized at negative edge of the clock.
6. The output signal **out** must be delivered for **64 cycle**, and **out_valid** should be **high** simultaneously.
7. The **out** signal should be **zero** when **out_valid** is low.
8. The **out_valid** cannot overlap with **in_valid** at any time.
9. **Please follow the parameter TA set, or you might fail in this lab.**
10. **You need to worry about infinity in the calculation process !!**

Specifications

1. Top module name: F_ATTN (File name: F_ATTN.v)
2. **You have to check an error under 10e-7 for the result after converting to float number. If the error is higher than the value, you will fail this lab.**
3. **It is asynchronous reset and active-low architecture. If you use synchronous reset (considering reset after clock starting) in your design, you may fail to reset signals.**
4. The reset signal (rst_n) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
5. The **out** should be reset after your **out_valid** is pulled down.
6. The execution latency is limited in **1000 cycles**. The latency is the clock cycles between the falling edge of the **in_valid** and the rising edge of the first **out_valid**.
7. The next inputs will come in the next 2 ~ 4 negative edge of the clock.
8. The area is limited in **4000000**. Also, the synthesis time should be less than **3 hours**.
9. You can adjust your clock period by yourself, but the maximum period is **50 ns**. The precision of clock period is 0.1, for example, 4.5 is allowed, 4.55 is not allowed.
10. The input delay is set to **0.5*(clock period)**.
11. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.

12. Check whether there is any wire / reg / submodule name called “error”, “fail”, “pass”, “congratulation”, “latch”, “DW_fp”, if you used, you will fail the lab.
13. The synthesis result of data type cannot include any latches.
14. After synthesis, you can check F_ATTN.area and F_ATTN timing. The area report is valid when the slack in the end of timing report should be **non-negative (MET)**.
15. **In this lab, you must use at least one IEEE floating point number IP from Designware. We will check it at F_ATTN.resource in 02_SYN/Report/.**

Block Diagram



Grading Policy

1. 2 demo: Score*0.7
2. Function Validity: 70%
3. Performance: 30 %
 - $\text{Area}^2 * \text{Computation time}$: 30%
 - $\text{Computation time} = \text{Latency} * \text{clock cycle time}$

Note

1. Please submit your files under 09_SUBMIT

1st_demo Due day: 10/12(Mon.) 12:00 pm

2nd_demo Due day: 10/14 (Wed.) 12:00 pm

If uploaded files violate the naming rule, you will get 5 deduct points.

2.
 - In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It means that the TA will demo your design under this clock cycle time:

3.

After that, you should check the following files under
09_SUBMIT/Lab04_iclabXXX/

- **F_ATTN_iclabXXX.v** (XXX is your account number)
- **filelist_iclabXXX.f**
- **clock_period_iclabXXX.txt** (ex: 15.0_iclab999.txt)

4. Template folders and reference commands:

- Check whether there is any wire / reg / submodule name called “error”, “fail”, “pass”, “congratulation”, “latch”, “DW_fp”, if you used, you will fail the lab.

01_RTL/ (RTL simulation) **./01_run_vcs_rtl**
02_SYN/ (Synthesis) **./01_run_dc_shell**

(Check if there is any **latch** in your design in **syn.log**)

(Check the timing of design in /Report/ **F_ATTN.timing**)

03_GATE / (Gate-level simulation) **./01_run_vcs_gate**
09_SUBMIT / (submit your files) **./00_tar**

./01_submit

./02_check

✖ **You should make sure the two clock period values identical in 00_TESTBED/Pattern.v &&
/02_SYN/syn.tcl:**

```
`define CYCLE_TIME       50.0  
`define SEED_NUMBER     28825252  
`define PATTERN_NUMBER 1000
```

```
#=====  
# (A) Global Parameters  
#=====  
set DESIGN "F_ATTN"  
set CYCLE 50
```

Example Waveform

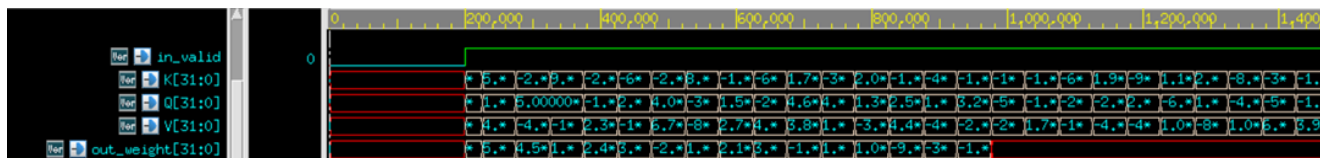


Fig1. Input waveform



Fig2. Output waveform

Some reminders

- Warning about IP shown in the figure below can be accepted in this lab.

```
Warning: Dont_touch on net '*cell*85/M_2scompl[3]' may be overridden by compile because it
is connected to generic logic. Use all_connected to see net
connections. (OPT-461)
Warning: Dont_touch on net '*cell*85/M_2scompl[4]' may be overridden by compile because it
is connected to generic logic. Use all_connected to see net
connections. (OPT-461)
Warning: Dont_touch on net '*cell*85/M_2scompl[5]' may be overridden by compile because it
is connected to generic logic. Use all_connected to see net
connections. (OPT-461)
Warning: Dont_touch on net '*cell*85/M_2scompl[6]' may be overridden by compile because it
is connected to generic logic. Use all_connected to see net
connections. (OPT-461)
Warning: Dont_touch on net '*cell*85/M_2scompl[7]' may be overridden by compile because it
is connected to generic logic. Use all_connected to see net
```

- If a timing violation message like the figure below occurs during gate-level simulation, you will fail this lab even if you pass the PATTERN.

```
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_0_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_6_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_5_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_4_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_3_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_2_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
"/RAID2/COURSE/iclab/iclabTA01/UMC018_CBDK/CIC/Verilog/fsa0m_a_generic_core_30.lib.src", 14852: Timing violation in TESTBED.I_SA.w_V_q_reg_16_1_
$setuphold( posedge CK:7354638, negedge D:7354429, limits: (211,-97) );
```